Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

Übung 06: VHDL - Asserts und Verification

M.Sc. Eike Trumann, M.Sc. Jasper Homann,

Prof. Dr.-Ing. Guillermo Payá Vayá

28. November 2025

1 Strukturelle Beschreibung, Funktionen

Die Parität einer Bitsequenz heißt gerade, wenn die Anzahl der Einsen im der Sequenz gerade ist, sonst ungerade. Gerade Parität soll durch eine 0 und ungerade Parität durch eine 1 kodiert werden.

Eine Möglichkeit die Parität einer Bitsequenz zu berechnen ist das XOR von allen Bits zu berechnen. Da XOR eine assoziative Funktion müssen die Bits nicht linear (eins nach dem anderen) kombiniert werden, stattdessen kann der kritische Pfad durch Verwendung einer Baumstruktur verkürzt werden.

Für diese Augabe soll eine solche Baumstruktur durch explizit mithilfe einer rekursiven Funktion implementiert werden.

Beschreiben Sie eine Funktion PARITY, die einen `std_ulogic_vector` entgegennimmt und die Parität berechnet. Dazu soll die Funktion sich rekursiv selbst mit je einer Hälfte der Eingabe aufrufen.

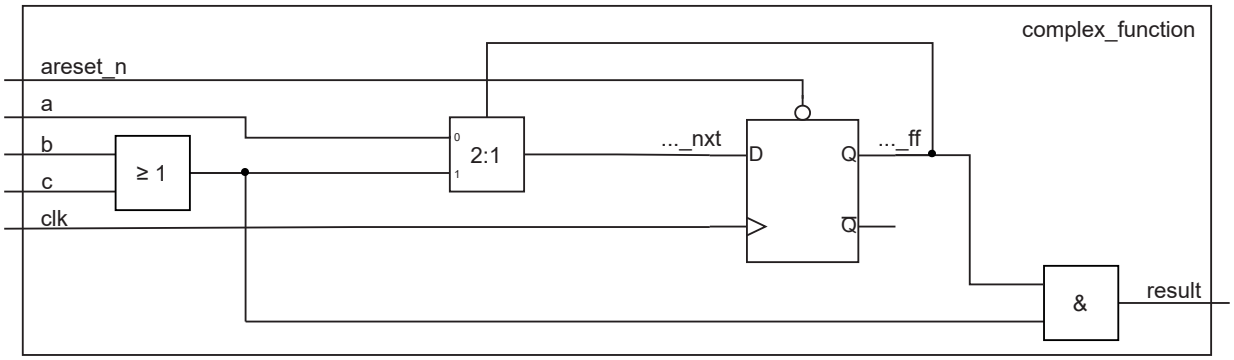
Hinweis: Verwenden Sie die Signalattribute `X'high`, `X'low` und `X'length`, um die Grenzen und die Länge eines Vektors zu bestimmen. Die folgende Tabelle gibt die Attributwerte Beispielhaft für ein Signal an, das als `signal X : std_ulogic_vector(7 downto 5);` deklariert ist.

Attribut	Wert
`X'high`	7
`X'low`	5
`X'length`	3

```
package recursive_parity_pkg is
        function PARITY (X : std_ulogic_vector) return std_ulogic;
end package;

package body recursive_parity_pkg is
        function PARITY (X : std_ulogic_vector) return std_ulogic is
                variable split_index : integer;
                variable half_parity1, half_parity2 : std_ulogic;
        begin
                -- TODO: Basisfall der Rekursion (length = 1) behandeln









                -- TODO: Berechnen an welchem Index der Vektor geteilt werden soll
                split_index :=









                -- TODO: Vektor teilen und Funktion rekursiv Aufrufen
                half_parity1 :=




                half_parity2 :=




                -- TODO: Paritäten kombinieren und Ergebnis zurückgeben
                return




        end function;
end package body;
```

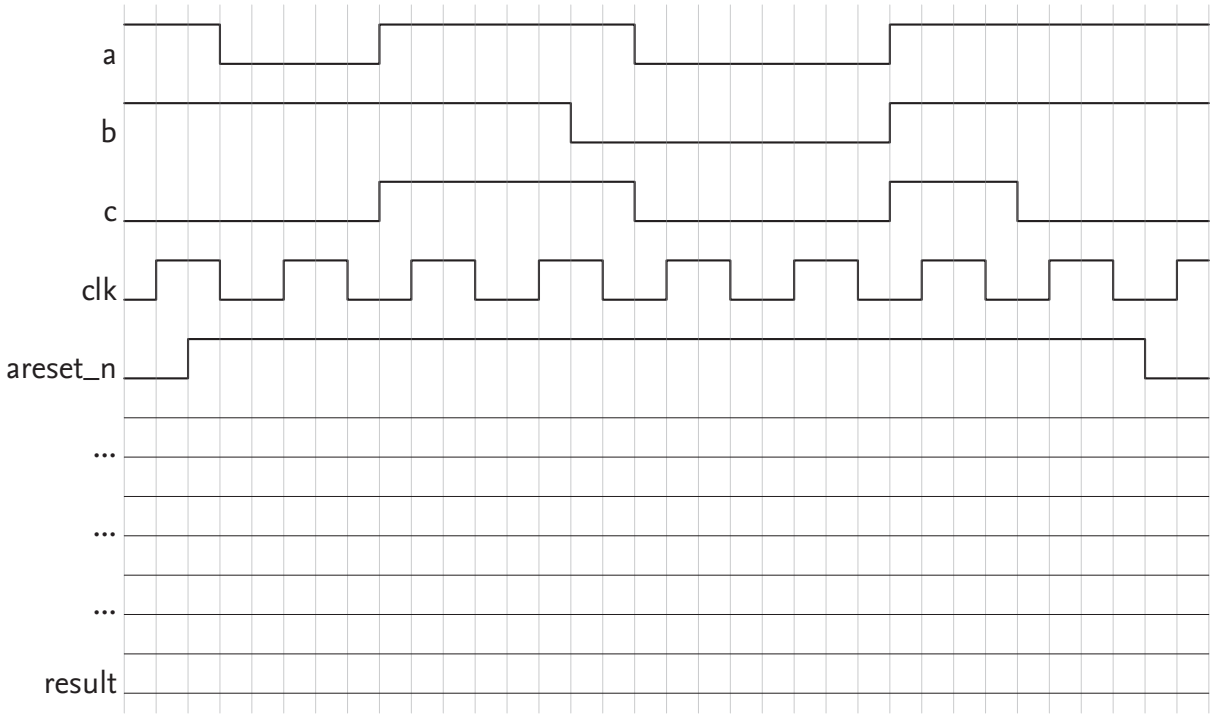
2 Implementierung Logik + Register

Beschreiben Sie folgende Schaltung in VHDL. Nutzen Sie zwei Prozesse (sequentiell und kombinatorische Logik getrennt).



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity complex_function is
port(
    clk         : in  std_ulogic;
    areset_n    : in  std_ulogic;
    a, b, c         : in  std_ulogic;
    result      : out std_ulogic
);
end entity complex_function;

architecture rtl of complex_function is
    signal                        : std_ulogic;
    signal                        : std_ulogic;
    signal                        : std_ulogic;
begin
    kombinatorisch: process(                                                )
    begin









    end process;

    sequentiell: process(                                                   )
    begin









    end process;


end architecture;
```

Vervollständigen Sie das Timing-Diagramm zu dieser Schaltung



3 Assert 1

Erstellen Sie zur vorigen Schaltung (Aufgabe (2)) eine Assert-Prüfung, ob der Ausgang '0' ist, bei anliegendem reset (asynchron).
Erstellen Sie außerdem eine assert-Prüfung, ob der Ausgang '0' ist, wenn (b or c) null ist.

```
architecture rtl of complex_function is
    ...
begin





    ...
end architecture;
```

Assert 2

Erstellen Sie für die Entity *bus_or_memory_entity* Asserts.

```
...
entity bus_or_memory_entity is
generic(
    NUM_ELEMENTS : natural := 16;
    DATA_WIDTH   : natural := 32;
);
port(
    clk          : in  std_ulogic;
    reset_n      : in  std_ulogic;
    rd_en, wr_en : in  std_ulogic;
    req          : in  std_ulogic;
    addr         : in  std_ulogic_vector(05             downto 00);
    data_i       : in  std_ulogic_vector(DATA_WIDTH - 1 downto 00);
    valid        : out std_ulogic;
    data_out     : out std_ulogic_vector(DATA_WIDTH - 1 downto 00)
);
end entity bus_or_memory_entity;
```

- Niemals darf gleichzeitig gelesen (rd_en) und geschrieben (wr_en) werden

```
architecture rtl of bus_or_memory_entity is
    ...
begin




...
```

- Das Signal 'addr' darf keine Elemente addressieren, die außerhalb der addressierbaren Elemente liegen (NUM_ELEMENTS)

```
...




...
```

- Das Signal 'valid' muss '1' sein, wenn im vorigen Takt ein Request erfolgte (req = '1'), ansonsten muss es '0' bleiben

```
...




end architecture;
```

4 Verification

Das Modul aus Aufgabe 2) soll nun getestet werden. Die Testbench soll dafür jeweils einen Takt lang **a**, **b** und **c** anlegen. Führen Sie im Anschluss an den Reset (siehe Template) die folgende Test-Sequenz durch. Ergänzen Sie in der Tabelle zunächst die erwarteten Ergebnisse.

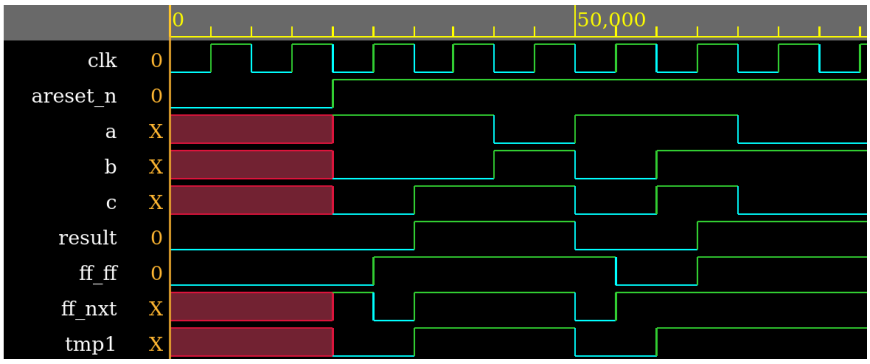
Test-Takt	a	b	c	result
0	1	0	0	
1	1	0	1	
2	0	1	1	
3	1	0	0	
4	1	1	1	
5	0	1	0	

Erzeugen Sie nun die Sequenz im Prozess *stimulus_process*. Fügen Sie passende Asserts und Fehlermeldungen hinzu.

```
entity tb is
end entity tb;

architecture rtl of tb is
    component complex_function is
      port(
        clk         : in  std_ulogic;
        areset_n     : in  std_ulogic;
        a, b, c : in  std_ulogic;
        result      : out std_ulogic
      );
    end component;

    signal clk      : std_ulogic := '0';
    signal areset_n : std_ulogic;
    signal a, b, c, result : std_ulogic;
begin
    -- clock generation --
    clk <= not clk after 5 ns;

    -- DUT --
    dut: complex_function
      port map(
        clk      => clk,
        areset_n => areset_n,
        a        => a,
        b        => b,
        c        => c,
        result   => result
      );
```

```
stimulus_process: process
  begin
    areset_n <= '0';
    wait for 20 ns;
    areset_n <= '1';

  end process;
end architecture;
```

Kurzfragen

Wie hoch ist die simulierte Taktfrequenz (Signal *clk*)?

Wie hoch ist die Coverage der durchgeführten Testsequenz? (Statement(Line)-, Branch- und Toggle-Coverage)